MOVIE TICKET BOOKING SYSTEM

CLI-BASED C PROGRAMMING PROJECT

Course Module: Software Development Fundamentels
Submission Date: 7/24/2026

TEAM MEMBERS & IDENTIFICATION

1. **N.P.N.H.Silva** (ID: **AS20250365**) — GitHub: **Nipu-2004**
2. **K.M.C.Denuwan** (ID: **AS20250611** — GitHub: **chamod-2004**
3. **E.A.C.N.Edirimanne** (ID: **AS20250592**) — GitHub: **charithediriphysical2004-cpu**
4. **S.S.Tharuka** (ID: **AS20250355**)—GitHub: **SandunAB12**

REPOSITORY INFORMATION

INTRODUCTION

The Movie Ticket Booking System is a Command-Line Interface (CLI) application developed in C. The core problem it solves is the inefficiency and potential human error associated with manual cinema seating and ticketing management.

This system automates the booking workflow. It allows users to view scheduled showtimes, visualize available and occupied seats using an interactive matrix display, reserve specific seats with real-time price calculations, and apply multi-tier discounts (such as Student, Senior, and Group variants). Additionally, the system provides operational utilities for administrators, including a robust booking search feature and a dynamic revenue generation report.

TEAM CONTRIBUTION SUMMARY

Member Name	Assigned Module	Specific Contribution
N.P.N.H.Silva	Architecture & Setup	Created global arrays, initialized structures, and set up the main GitHub repository.
K.M.C.Denuwan	Frontend & UI Design	Designed the CLI seat grid layout and handled showtime display formatting.
E.A.C.N.Edirimanne	Backend Logic & Pricing	Coded core seating reservations, validity boundary controls, and pricing updates.
S.S.Tharuka	Testing & Documentation	Built booking search lookup functions, executed validation test suites, and compiled reports.

DESIGN OVERVIEW

Our program is structured around parallel processing tracks using multi-dimensional array indices to manage memory seamlessly:

1. **Seating Tracks:** Managed using `seats_show[ROWS][COLS]` global matrices representing a 5-row by 10-column theater layout environment.
2. **Profiles Tracking:** Captured via `names_show[ROWS][COLS][50]` three-dimensional character grids dynamically binding customer identity blocks up to 50 text positions deep.
3. **Ledger Systems:** Driven by `price_show[ROWS][COLS]` floating decimal containers maintaining strict purchase values for live system metrics.

`main()` (Launches the central execution menu loop)

- `viewShowtimes()` & `viewSeatMap()` (Handles text layouts and grid matrices visualizers)
- `bookSeats()` & `cancelBooking()` (Performs seat data validation and state rollback updates)
- `searchBooking()` & `viewRevenueReport()` (Manages identity pattern lookups and global validation checks)

This section presents chronological visual terminal execution captures of the running Movie Ticket Booking application

Figure 1: Main Menu Dashboard Interface and Empty Seat Map Layout (Option 2 View).

This screenshot demonstrates the central Command-Line Interface (CLI) navigation menu featuring options 1 through 7. It also captures the automated execution of the 5x10 seating matrix display (Option 2), proving that all configuration parameters initialize correctly with blank seat statuses indicated by dots (.)

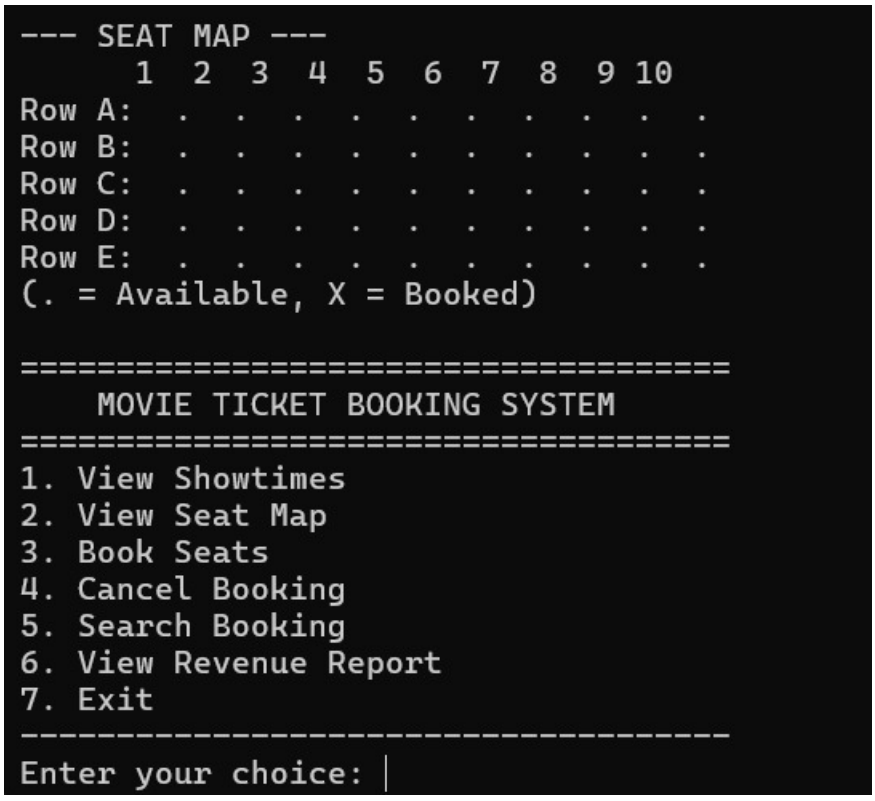
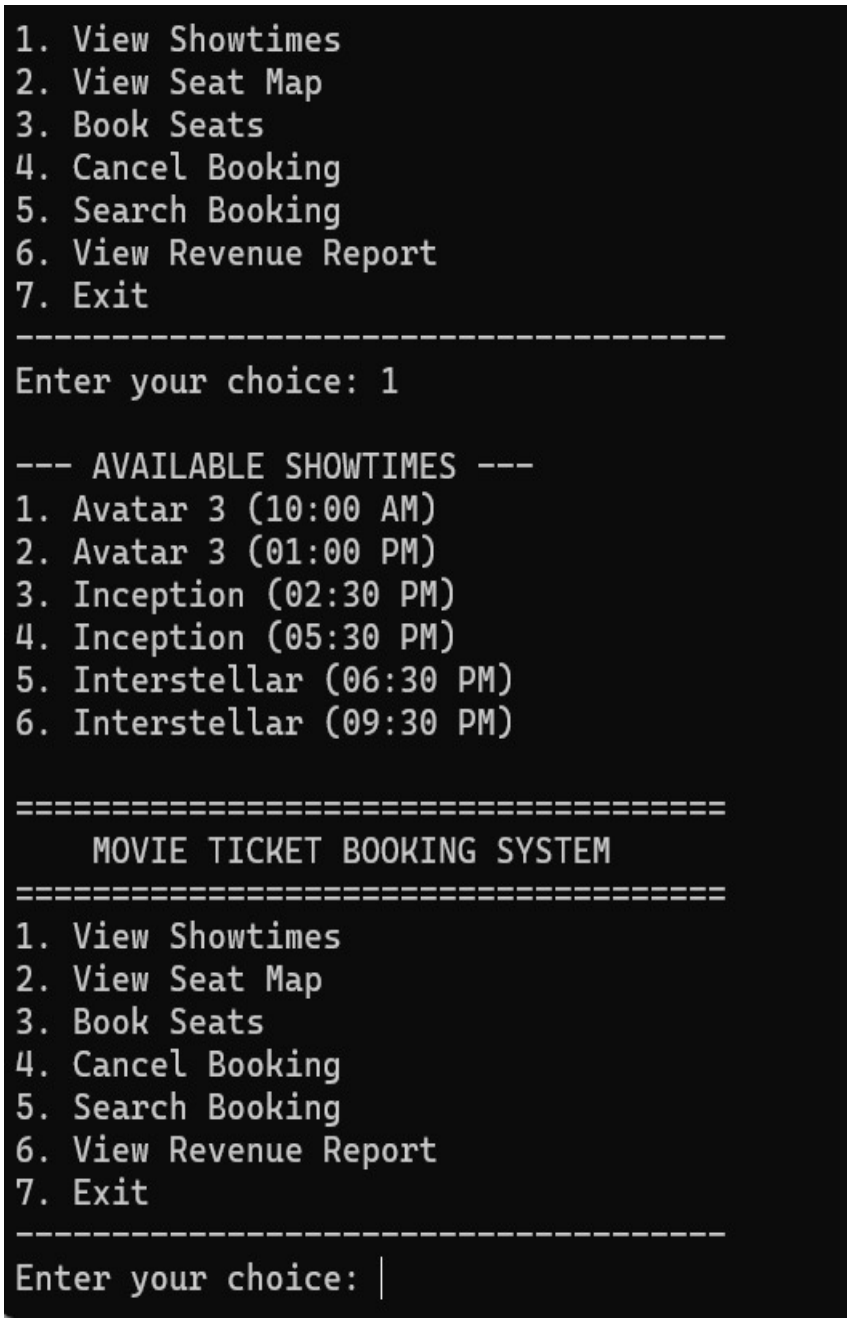


Figure 2: Movie Showtimes List Display (Option 1 Execution).

This screenshot displays the output when Option 1 is selected. The system successfully extracts and lists all 6 hardcoded movie screening schedules (Avatar 3, Inception, and Interstellar) alongside their respective timing configurations.



CHALLENGES & HOW THEY WERE SOLVED

1. **Challenge: Input Buffer Crashing**
   Description: Entering non-integer string characters into numeric input choices caused infinite loop system failures inside the terminal scanner.
   Solution: Resolved by incorporating a sequential character clearing mechanism using a standard checking sweep loop (`while(getchar() != '\n');`) after every input capture.
2. **Challenge: Memory Overlap During Branch Management**
   Description: Simultaneous remote directory code pushes generated complex line alignment conflicts inside shared text structures.
   Solution: Resolved by enforcing isolated developer work paths (`feature branches`) on GitHub and checking line alignments via automated Git validation steps before running mergers.

GITHUB COLLABORATION SUMMARY

Our repository structure followed strict version control practices. Every team member checked out specialized functional development tracking spaces (`feature branches`) to safeguard steady production deployment.

QUALITY ASSURANCE (QA) & TEST LOG

Test ID	Test Scenario	Test Input	Expected Output	Status
TC_001	Double Booking Prevention	Book occupied seat A1	"Seat already booked!"	**PASSED**
TC_002	Boundary Value Validation	Enter seat row 'Z'	"Invalid seat position!"	**PASSED**
TC_003	Cancellation Safety Check	Cancel empty seat B2	"Seat is already vacant..."	**PASSED**
TC_004	Invalid Menu Selection	Enter menu option '9'	"Invalid choice! Please try again."	**PASSED**
TC_005	Empty Search Result	Search unbooked seat	"Seat is currently available..."	**PASSED**

GITHUB COLLABORATION SUMMARY

During our group project development process, an oversight occurred in our initial code setup, where the program failed to calculate individual ticket values and multi-tier discount categories dynamically based on specific seat selections and demographics.

To rectify this error promptly, our team collective re-evaluated the logical flaws, rewrote the entire backend calculation modules to ensure precise pricing tracking, and successfully pushed the corrected final code to our remote GitHub repository as a new standalone update. Consequently, this final submission report, alongside all documented walkthrough screenshots and functional test cases, is entirely based on this newly pushed, fully verified master source code.